24. API Mimarisi: REST, GraphQL ve gRPC Karşılaştırması

Hangi Protokolü Ne Zaman Seçmelisin? OpenAPI Şemalarıyla Tip Güvenliği
Bölüm 4 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"REST öldü mü? GraphQL her derde deva mı yoksa gRPC mikroservislerin gizli süper gücü mü?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 REST basitliği ve HTTP önbelleklemesiyle kamusal API'lerin (Public API) kralıdır. - 📌 GraphQL, mobil uygulamalarda aşırı veri çekme (Over-fetching) ve yetersiz veri çekme (Under-fetching) sorununu çözer. - 📌 gRPC (Protocol Buffers), mikroservisler arasında ikili (binary) formatta HTTP/2 ile ışık hızında haberleşme sağlar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Farklı ülkelere mektup gönderdiğinizi düşünün.

1. **REST:** Standart bir posta mektubudur. Zarfın üstünde adres yazar (URL), içine standart kağıt konur (JSON). Dünyadaki her postacı bunu anlar.
2. **GraphQL:** Kişiye özel alışveriş sipariş formudur. Forma tam olarak *"Bana sadece kırmızı kazağın bedenini ve fiyatını getir, kumaş bilgisini getirme"* yazarsınız; kurye tam istediklerinizi getirir.
3. **gRPC:** İki askeri üs arasındaki kriptolu fiber optik kablodur. İngilizce veya Türkçe konuşmaz; ikili (binary 0 ve 1) şifreli sinyallerle mikrosaniyede haberleşir.

Bir e-ticaret sitesi yapıyorsanız REST veya GraphQL seçersiniz. Ancak arka planda binlerce finansal işlemi konuşturan sunucularınız varsa gRPC seçersiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Protokol Karar Ağacı:

```
[Yeni Bir API Geliştirilecek]
             |
             ├── Kamuya açık genel API veya standart CRUD mu?
             |   └──> REST API (OpenAPI / Swagger ile)
             |
             ├── Karmaşık mobil ekranlar ve ilişkili veriler mi var?
             |   └──> GraphQL (Apollo / Yoga)
             |
             └── İç mikroservisler arası yüksek hızlı veri transferi mi?
                 └──> gRPC (Protocol Buffers)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: API Mimarisi ve Entegrasyon Uzmanı
Bir ödeme platformu için 'Müşteri İşlemleri API'si tasarlıyoruz.

## GÖREV:
1. REST, GraphQL ve gRPC arasındaki farkları; gecikme (latency), ağ yükü (payload size) ve önbellekleme (caching) açısından karşılaştır.
2. Bu sistem için OpenAPI 3.0 (Swagger) uyumlu bir `POST /v1/payments` endpoint şeması (YAML) hazırla.
3. Bu şemadan TypeScript client tiplerinin nasıl otomatik türetileceğini (Contract-First) göster.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```yaml
# Kötü / Dokümante Edilmemiş REST: Tip yok, hata şeması belirsiz
/api/pay:
  post:
    description: Para ode
    responses:
      200:
        description: Basarili
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```yaml
openapi: 3.0.3
info:
  title: Core Payments API
  version: 1.0.0
paths:
  /v1/payments:
    post:
      summary: Yeni bir ödeme işlemi başlatır
      operationId: processPayment
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PaymentRequest'
      responses:
        '201':
          description: Ödeme başarıyla onaylandı
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/PaymentResponse'
        '400':
          description: Geçersiz parametreler veya yetersiz bakiye
components:
  schemas:
    PaymentRequest:
      type: object
      required: [amount, currency, sourceToken]
      properties:
        amount:
          type: number
          format: float
          minimum: 0.01
        currency:
          type: string
          enum: [USD, EUR, TRY]
        sourceToken:
          type: string
          example: "tok_visa_1234"
    PaymentResponse:
      type: object
      properties:
        paymentId:
          type: string
          format: uuid
        status:
          type: string
          enum: [SUCCEEDED, PENDING, FAILED]
```

💡 **Kod Analizi:** Contract-First yaklaşımında bu YAML dosyası yazıldıktan sonra `openapi-typescript` aracıyla hem frontend hem backend için tek tuşla sıfır hatayla TypeScript tipleri ve SDK üretilir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Basit bir blog sitesi için GraphQL kurup gereksiz N+1 sorgu karmaşasıyla boğuşmak.
  - **Çözüm:** İhtiyaç yoksa REST ile başlayın; karmaşık veri grafikleri oluştuğunda GraphQL'e geçin.
- **Tuzak:** REST API'de HTTP durum kodlarını (200, 400, 404, 500) görmezden gelip her hatada 200 OK içinde `{ error: true }` dönmek.
  - **Çözüm:** HTTP standartlarına uyun; hataları doğru durum kodlarıyla iletin.
- **Tuzak:** API sözleşmesini (OpenAPI) elle yazıp koddan koparmak.
  - **Çözüm:** Şemayı ya koddan otomatik türetin ya da şemayı tek gerçeklik kaynağı kabul edip kodu şemadan üretin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 24: Modern API Mimarisi - REST, GraphQL ve gRPC*

- 🖥️ **Slayt 1:** API Protokolleri Karşılaştırması: REST vs GraphQL vs gRPC.
- 🖥️ **Slayt 2:** Over-fetching ve Under-fetching problemlerini anlamak.
- 🖥️ **Slayt 3:** Contract-First Yaklaşımı: OpenAPI ile Tip Güvenliği.
- 🖥️ **Slayt 4:** Bölüm 4 Özeti: Yazılım Mimarisi ve Tasarım Kalıpları.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir OpenAPI YAML dosyasından `openapi-typescript` CLI aracıyla 5 saniyede TypeScript tipleri üretin. Bu tipleri hem Express rotasında hem de fetch çağrısında kullanarak tam tip güvenliğini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki en önemli 1 REST endpoint'i için Swagger / OpenAPI 3.0 uyumlu şema tanımı yazın veya Zod kullanarak şema türetin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: GraphQL'in geleneksel REST API'ye göre en belirgin avantajı nedir? - A) Asla hata vermemesi - B) İstemcinin (Frontend/Mobil) tam olarak ihtiyaç duyduğu alanları talep edebilmesi (Over-fetching ve Under-fetching'i önlemesi) - C) Sadece Python'da çalışması - D) CSS yüklemesini hızlandırması > **Doğru Yanıt:** B > **Açıklama:** GraphQL istemcinin tek bir sorguyla sadece istediği alanları ve iç içe ilişkili nesneleri tek istekte alabilmesini sağlar.

Soru 2: İç mikroservisler arasında yüksek hız ve düşük gecikme gerektiğinde gRPC'nin tercih edilmesinin temel teknik sebebi nedir? - A) Metin tabanlı JSON yerine Protocol Buffers (ikili veri) kullanması ve HTTP/2 üzerinden çalışması - B) Şifre gerektirmemesi - C) Sadece Google tarafından kullanılması - D) Veritabanı istememesi > **Doğru Yanıt:** A > **Açıklama:** gRPC, binary serialization (Protocol Buffers) ve HTTP/2 multiplexing sayesinde JSON'a kıyasla çok daha küçük paket boyutu ve yüksek hız sunar.